

Typescript

TSConfig

```
PS C:\Users\yun\Desktop\TypeScript> tsc --init  
  
Created a new tsconfig.json  
  
You can learn more at https://aka.ms/tsconfig
```

tsconfig.json

TSConfigファイルはTypescriptをJavascriptに変更することを定義するファイルである。

ファイルを含める

include

ファイルを含める最も一般的な方法でtsconfig.jsonのinclude属性を使う。

```
{  
  "include" : ["src"]  
}
```

exclude

excludeはincludeのスタートリストでファイルを削除する作業 だけだ。

```
{  
  "exclude" : ["**/external", "node_modules"],  
  "include" : ["src"]  
}
```

タイプ検査

strict

strict modeを使うために

```
PS C:\Users\yun\Desktop\TypeScript> tsc --strict
```

tsc —strictを入力すると

```
// Recommended Opti  
"strict": true,
```

compile optionにstrict : trueが生成される。

noImplicitAny

```
"noImplicitAny": false,
```

noImplicitAnyは暗黙的なanyタイプを禁止する。

このようにfalseをしてコードを入力すると

```
✓ function add(a, b) {  
    return a+b;  
}
```

エラーはない。

しかし、trueをすると

```
"noImplicitAny": true,
```

```
function add(a, b) {  
    return a+  
}
```

Parameter 'a' implicitly has an 'any' type. ts(7006)
(parameter) a: any
View Problem (Alt+F8) Quick Fix... (Ctrl+.) Fix (Ctrl+I)

エラーになる。

strictNullChecks

strictNullChecksはタイプを厳しく検査することである。

```
"strictNullChecks": false,
```

strictNullChecksをfalseにすると

```
const a: number = null;  
const b: number = undefined;  
const c: number = '3';
```

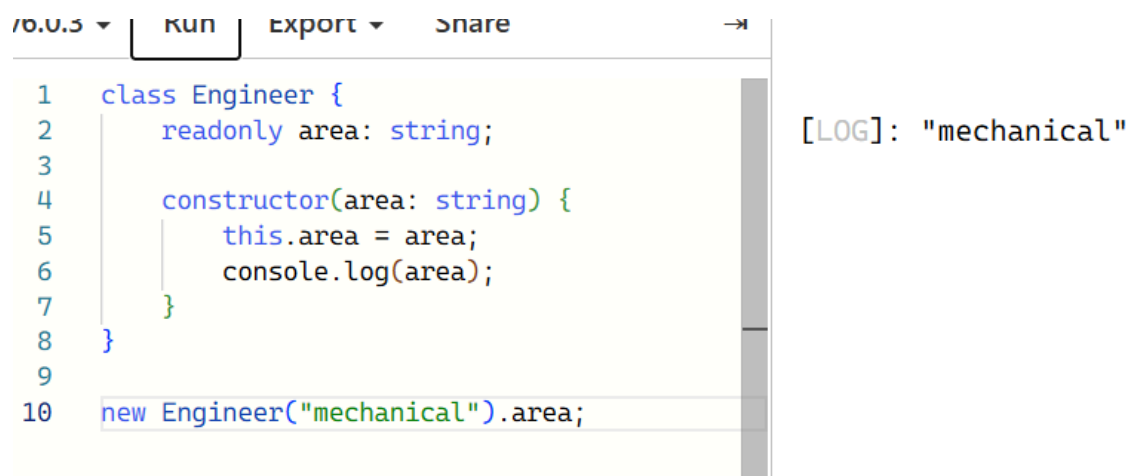
このようにタイプが間違えることになる。

```
"strictNullChecks": true,
```

trueにすると

```
const a: number = null;  
const b: number = undefined;  
const c: number = '3';
```

全部エラーになる。



```
1 class Engineer {  
2   readonly area: string;  
3  
4   constructor(area: string) {  
5     this.area = area;  
6     console.log(area);  
7   }  
8 }  
9  
10 new Engineer("mechanical").area;
```

[LOG]: "mechanical"

Javascriptのクラスはこのようにコンストラクでタ引数を受けてすぐにクラス属性に割り当てることが一般的である。

```
v6.0.3 ▾ Run Export ▾ Share →
1 class Engineer {
2   constructor(readonly area: string) {
3     console.log(area);
4   }
5 }
6
7 new Engineer("mechanical").area;
```

```
[LOG]: "mechanical"
[LOG]: "mechanical"
```

Typescriptは引数にreadonly, public, protected...を付けて短くすることができる。

列挙型

Typescriptはリテラル値をもつオブジェクトを生成するenumを提供する。

```
enum StatusCode {
  InternalServerError = 500,
  NotFound = 404,
  OK = 200,
}

console.log(StatusCode.InternalServerError);
```

```
[LOG]: 500
```

このように列挙型を使用する。